

Enterprise Capstone & SRE Production Readiness

Category: **Site Reliability Engineering & Production Excellence** | Platform Standard: **Linux 6+ / Modern Cloud SRE**

1. Domain Architectural Overview

This official module provides comprehensive engineering depth and runnable syntax for **Enterprise Capstone & SRE Production Readiness**. It is strictly scoped to this domain to prevent knowledge cross-contamination. Engineers mastering this domain acquire hands-on proficiency from foundational setup to production-grade incident triage and resilience.

2. Core Concepts & Production Principles

- **SLI, SLO, SLA & Error Budget Mechanics:** Service Level Indicators (SLIs) measure real performance. Service Level Objectives (SLOs) set reliability targets agreed upon by engineering. Service Level Agreements (SLAs) are business contracts with financial penalties. $\text{Error Budget} = 100\% - \text{SLO}$.

Production Application: When an error budget is exhausted, product feature deployments are frozen and all engineering capacity shifts to reliability and technical debt remediation.

- **Production Readiness Review (PRR) Framework:** A formal audit conducted before onboarding or launching any new service into production, covering architecture, observability, scalability, failure modes, security, and operational runbooks.

Production Application: Guarantees that no service goes live without health checks, automated alerts, resource limits, on-call runbooks, and graceful degradation paths.

- **Blameless Postmortems & 5 Whys Root Cause Analysis:** A culture and structured post-incident process assuming human operators do their best with the tools available. Focuses on systemic flaws, tooling gaps, and process deficiencies rather than individual blame.

Production Application: Drives continuous systemic improvement: turns every major production failure into high-priority engineering action items (P0/P1 fixes).

- **GitOps Architecture & Continuous Reconciliation:** Operating model where Git is the single source of truth for desired infrastructure and application state. An automated agent (ArgoCD/Flux) continuously compares live state to Git state and automatically remediates drift.

Production Application: Eliminates configuration drift and prevents manual cluster tampering; all production changes are auditable Git pull requests.

3. Exhaustive Command & Syntax Reference

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
argocd app sync --prune --timeout 300 <code>argocd app sync [options]</code>	Trigger manual synchronization of a GitOps application in ArgoCD, pruning obsolete cluster resources and waiting up to 5 minutes for health validation. Options: [{flag: '--prune', 'description': 'Delete resources from the cluster that no longer exist in the Git source repository.'}, {flag: '--timeout 300', 'description': 'Maximum seconds to wait for application resources to reach healthy state.'}]	Run during planned production releases to synchronize Kubernetes clusters with audited Git repository commits. Pitfall: Syncing with <code>--prune</code> without verifying whether obsolete resources contain stateful persistent volumes.
velero backup create prod-k8s-backup-2026 --include-namespaces production --snapshot-volumes <code>velero backup create [options]</code>	Generate an atomic, consistent backup of all Kubernetes resource manifests and persistent storage volume snapshots across the production namespace. Options: [{flag: '--include-namespaces', 'description': 'Scope backup strictly to specified namespaces.'}, {flag: '--snapshot-volumes', 'description': 'Trigger cloud provider EBS/PersistentDisk storage snapshots for all persistent volume claims.'}]	Run daily via automated Velero schedules to ensure Recovery Point Objectives (RPO < 1 hour) are met. Pitfall: Taking backups without validating storage snapshot integration, leading to empty volume backups.
chaos-mesh apply -f network-latency-experiment.yaml <code>chaos-mesh apply -f</code>	Inject controlled network latency and packet loss into microservices to validate circuit breakers and resilience timeouts during a scheduled GameDay. Options: [{flag: '-f', 'description': 'YAML manifest defining target pods, latency duration (e.g. 200ms), and blast radius.'}]	Execute during scheduled GameDay exercises with on-call engineers actively monitoring alerts and dashboards. Pitfall: Running chaos experiments in production without defining an immediate kill switch or blast radius cap.
aws route53 change-resource-record-sets --hosted-zone-id Z123456789 --change-batch file://failover-dns.json <code>aws route53 change-resource-record-sets [options]</code>	Execute emergency DNS traffic failover to redirect customer ingress traffic from a primary region to a secondary disaster recovery region. Options: [{flag: '--hosted-zone-id', 'description': 'Route53 public hosted zone identifier.'}, {flag: '--change-batch', 'description': 'JSON payload declaring UPSERT record changes with target load balancer IPs.'}]	Triggered when an entire cloud availability zone or region suffers severe physical hardware or network isolation. Pitfall: Using high TTL values on DNS records, causing customer ISP DNS caches to retain obsolete routes for hours.

Command & Syntax	Purpose & Options	Production Usage & Common Pitfall
<code>crane export ghcr.io/org/monolith:v1.0.0 /tmp/monolith.tar</code> <code>crane export</code>	Export a remote container image filesystem directly into a tarball for offline security inspection and forensic audit without running a Docker daemon. Options: [{flag: ", 'description': 'Target container image location in remote registry.'}, {flag: ", 'description': 'Path to local output tarball.'}]	Run in automated security analysis sandboxes to inspect container file contents without running untrusted code. Pitfall: Attempting to run untrusted container images locally to inspect contents, risking host compromise.

4. Security Hardening & Anti-Patterns

Security Mandates: Always operate under least-privilege principles. Restrict credentials, mask secrets in build outputs, avoid wildcard IAM permissions, and validate inputs before running commands.

Critical Anti-Pattern: Avoid manual drift ('click-ops' in cloud consoles or unversioned hotfixes in staging/production). Every change must be declared in code and reviewed via CI/CD.